

SHIV NADAR UNIVERSITY CHENNAI

School of Engineering

CS3807 – Deep Learning Laboratory

Experiment – 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Name : Srivatsan S

Register No. : 24011101093

Degree & Branch : B.Tech Artificial Intelligence & Data Science

Class / Section : AIDS – B

Semester : V

Subject Code : CS3807

Subject Name : Deep Learning Laboratory

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: Srivatsan S

Reg No.: 24011101093

Experiment 3 Implementation of Convolutional Neural Networks (CNNs) for Image Classification

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

- N : Input Size
- F : Filter Size
- P : Padding
- S : Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,

Total trainable parameters = 448

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

graphicx epstopdf placeins

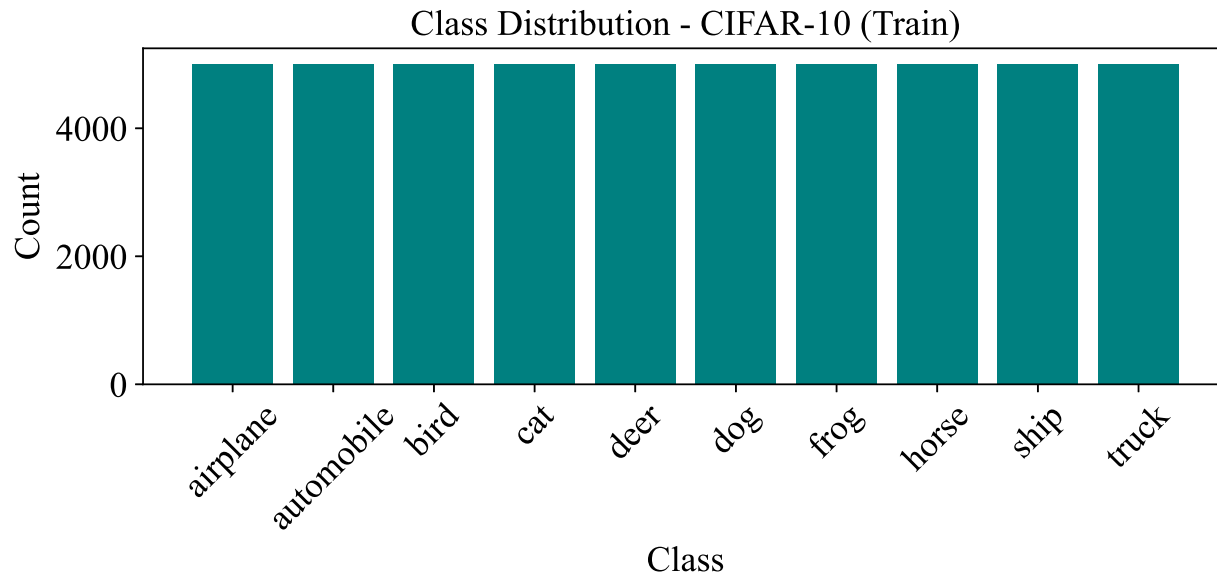
9. Generated Plots with Interpretation

9.1 Sample Images

Sample Training Images

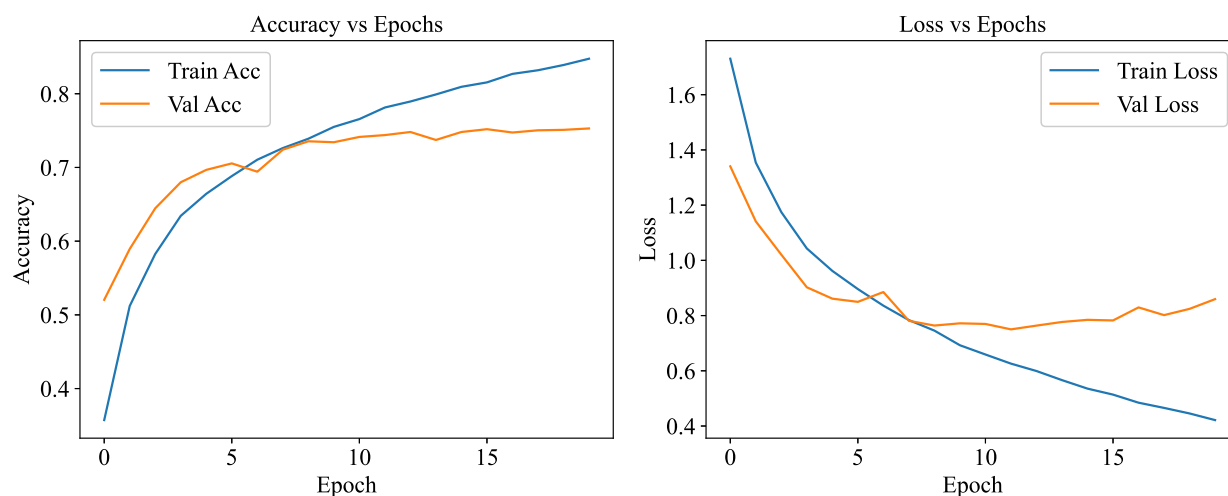


9.2 Class Distrubution



Interpretation: The dataset is completely balanced when it comes to its classes as all the classes has the same number of datapoints

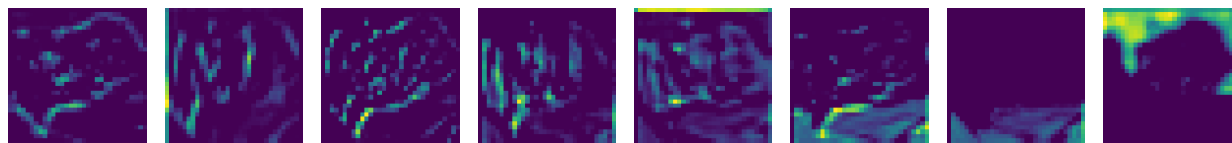
9.3 Training Curves



Interpretation: Both Training and validation Accuracy rises steadily upto epoch 7 and further training accuracy kept rising till epoch 20 whereas validation accuracy plateaued over a small range. Similarly both training and validation loss steadily cascades upto epoch 8-10 further validation loss stagnates at a small range whereas training loss steadily decreased. We can attribute this behaviour to the fact that the model had started to overfit around epoch 8-10 .

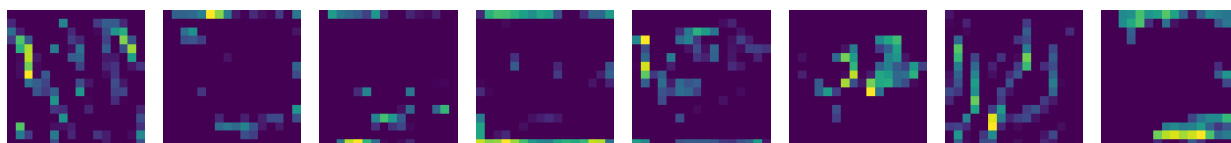
9.4 Feature Maps

Feature Maps - conv1



Interpretation: The feature map contains spatially interpretable templates and silhouettes of edges , curves , colour contrasts and textures

Feature Maps - conv2



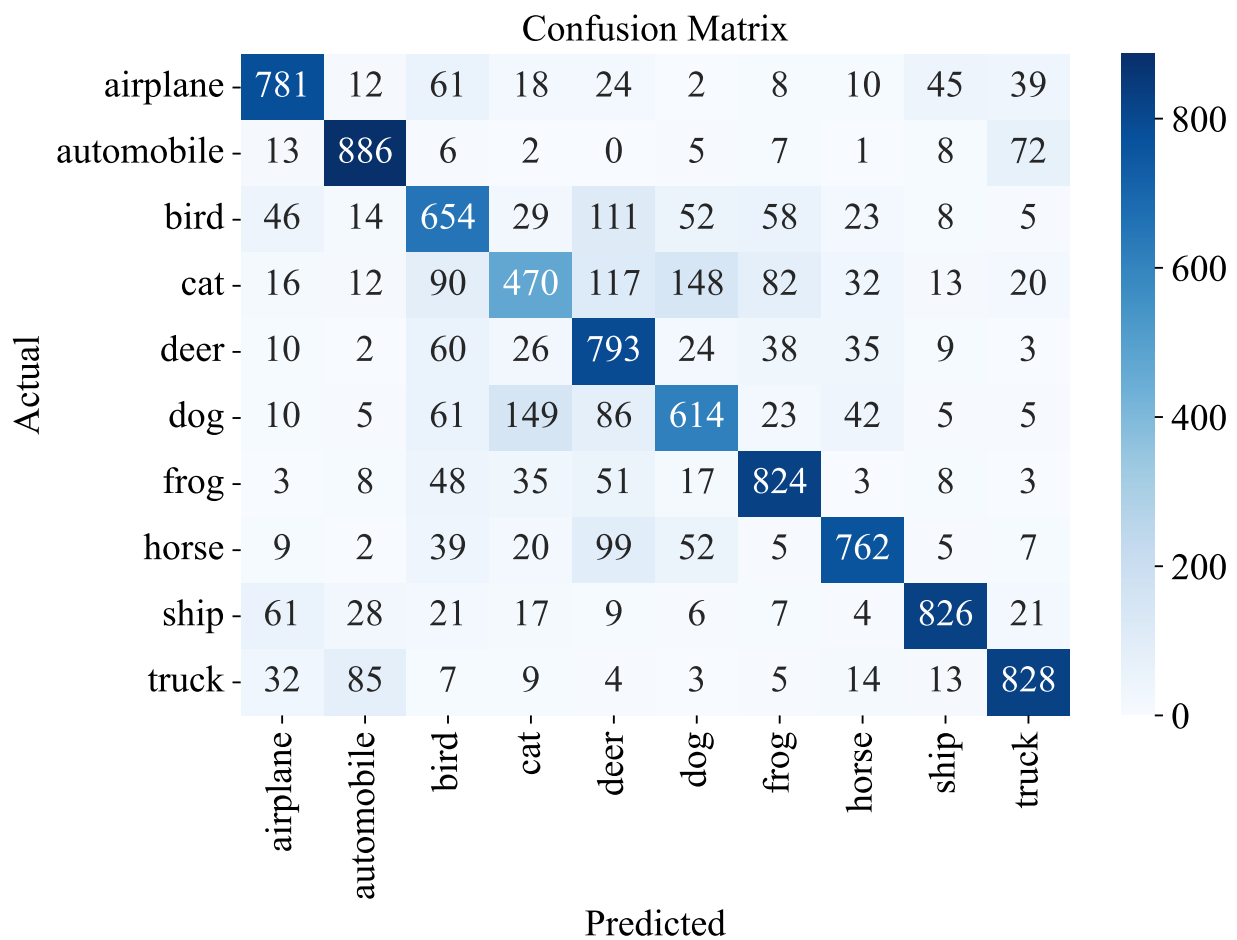
Interpretation: This layer of feature maps are much more abstract than that of layer 1 because of the pooling happening after the first convolution

Feature Maps - conv3



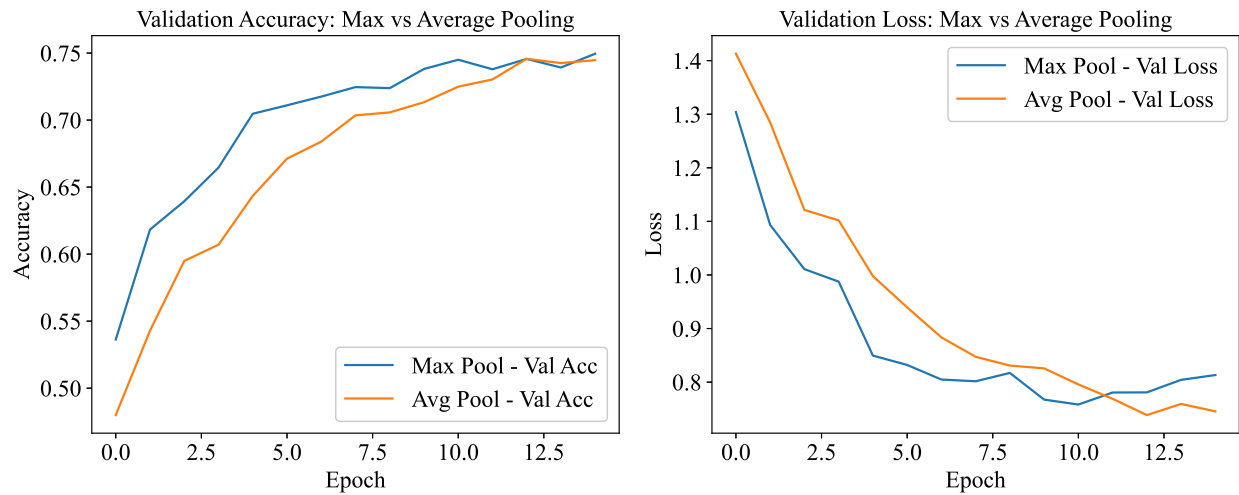
Interpretation: All of the feature maps in this level is entirely sparse and almost blank . This can be attributed to the sampling size after the three layers and these activations representing highly abstract class specific patterns .

9.5 Confusion Matrix



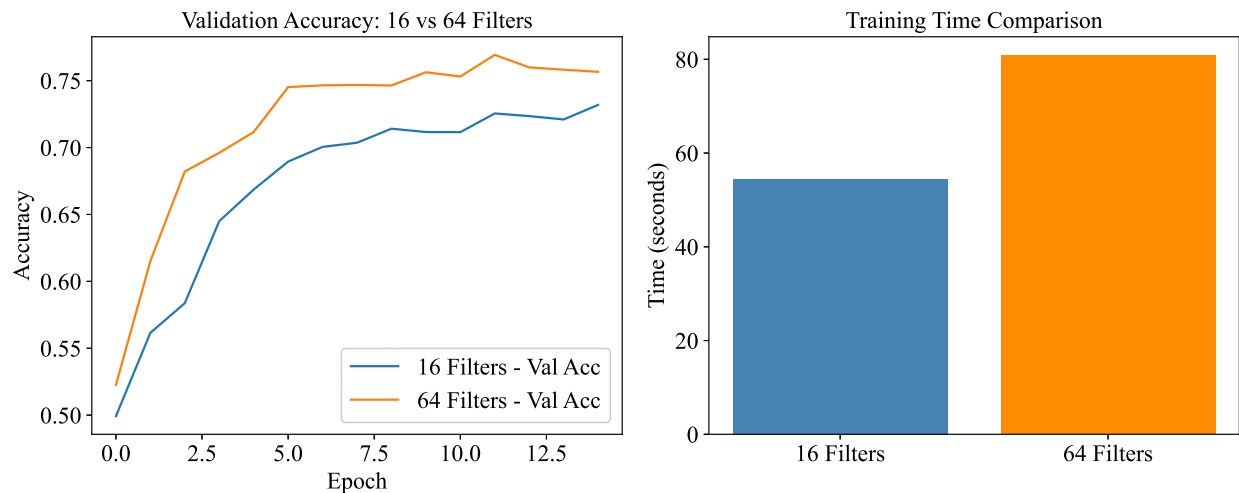
Interpretation: The model performs well and had correctly classified (Diagonal elements are strong i.e classified as per its correct class) and for visually distinct classes and such as vehicles and ships but, struggles when it comes to visually similar classes such as cats and dogs , bird, deer and frogs and also automobile and trucks .

9.6 Pooling Comparison Max vs Avg



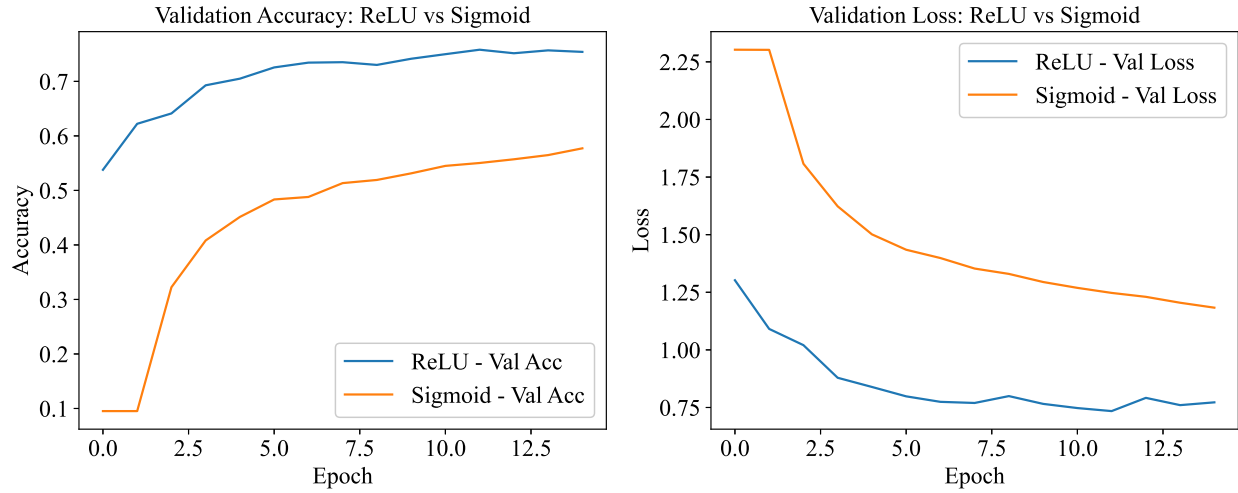
Interpretation: Max pooling converges faster than average pooling whereas average pooling has slow early convergence but can generalize better and both of the validation accuracy and around the same range .

9.7 Filters Comparison 16 vs 64



Interpretation: Increasing the model's filter size increased the model's representational capacity but increased the training time substantially

9.8 Activation Function Comparison Relu vs Sigmoid



Interpretation:

Sigmoid did not start learning for the first epoch and flattens out at a lower accuracy rate than Relu which outperformed Sigmoid similarly in the validation loss the sigmoid is much more elevated than Relu signifying that Relu performs better than sigmoid =====

9.9 Output Size Calculation

Given: Input size $N = 64$, Kernel size $K = 5$, Stride $S = 2$, Padding $P = 2$.

$$\begin{aligned} \text{Output size} &= \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1 \\ &= \left\lfloor \frac{64 + 2(2) - 5}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32 \end{aligned}$$

$$\boxed{\text{Output size} = 32 \times 32}$$

9.9 Trainable Parameters Calculation

Given: Number of filters = 64, Kernel size = 3×3 , Input channels (RGB) = 3.

$$\begin{aligned} \text{Params} &= (K_h \times K_w \times C_{in} + 1) \times N_{filters} \\ &= (3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 28 \times 64 \end{aligned}$$

$$\boxed{\text{Trainable Parameters} = 1,792}$$

10. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

Metric	Recorded Result
Training Accuracy	0.8475 (84.75%)
Testing Accuracy	0.7438 (74.38%)
Precision	0.7453
Recall	0.7438
F1-score	0.7420
Number of Parameters	356,810

11. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?

For a fully connected layer each neurons takes hundreds of thousands as input and the pixel is treated independently . Whereas Convolution uses small kernels and also local connectivity which cuts parameters

2. How does stride affect the feature map size?

Larger the stride the more positions the skips and fewer output position and samller feature maps

3. What is the role of padding?

Padding adds extra pixels aroud the border to prevent shrinking and preserves border information

4. Why is pooling used?

Pooling downsamples feature maps to reduce parameters and computaiton , meaning small shifts in the object position does not significantly chanfe the output i.e translational invariance

5. How do feature maps represent image characteristics?

Feature maps are outputs of convolution filters applied across the image . Layers produce maps showing templates such as edges and colour contrasts in the early layers but as we progress down the convolutions the maps combine abstract and class specific patterns and also shrink the pooling .

6. Why do CNNs require fewer parameters than MLPs?

Due to weight sharing , local receptive fields

12. SOURCE CODE

<https://github.com/sriwhatsnext/dllab/tree/main/Week-03-CNN>

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.